FINAL PROJECT REPORT

A Passenger Requests and Generates a Metro Ticket Through the Service Catalog

Name: Boyapati Rishi

Team ID: Group

Platform: ServiceNow

Project Version: 1.0

Date: 20 September 2026

ServiceNow Metro Ticket Generating System

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

2.3 Brainstorming

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

3.2 Solution Requirement

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN

4.1 Problem Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

7. RESULTS

7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

1. INTRODUCTION

1.1 Project Overview

The project, "A passenger requests and generates a metro ticket through the Service Catalog," is a ServiceNow-based automation solution designed to give passengers a standardized, paperless way to obtain a metro ticket. The solution replaces counter-based booking and manual fare calculation with a structured Service Catalog request and an automated ticket-generation lifecycle.

The passenger submits the journey details through a dedicated catalog item named Metro Ticket Request. The request generates the related Request (REQ) and Requested Item (RITM) records. A configured Flow Designer flow validates the journey, calculates the fare from the station data, and routes group bookings (more than five passengers) to the Station Manager for approval. Once the request is valid and approved, the flow creates a uniquely numbered Metro Ticket record, notifies the passenger by email, and closes the Requested Item.

The project also uses Business Rules for backend automation (journey validation, fare calculation, and ticket validity), a Scheduled Script Execution to expire old tickets, custom ServiceNow tables for station and ticket data, Access Control Lists (ACLs) for security, and Update Sets for controlled migration of configuration between instances.

1.2 Purpose

The purpose of the project is to make metro ticket booking simpler, faster, more transparent, and easier to track. The solution provides a consistent request form, calculates the fare automatically, generates a digital ticket with a unique ticket number, and maintains the complete ticket lifecycle within ServiceNow.

- Provide a standardized Service Catalog item for metro ticket requests.
- Capture Source Station, Destination Station, Journey Date, Number of Passengers, Ticket Type, and Payment Mode.
- Create and relate REQ, RITM, approval, and Metro Ticket records.
- Calculate the fare automatically from station data using Business Rules.
- Automate group-booking approval and ticket generation using Flow Designer.
- Notify the passenger with the ticket number and support ticket tracking, automatic expiry, and controlled deployment using Update Sets.

2. IDEATION PHASE

2.1 Problem Statement

Passengers who need to travel by metro often depend on counter-based booking, where they may face long queues, manual fare calculation, and no digital record of the ticket. When a group needs several tickets, the booking may also require coordination with station staff, leaving the passenger with limited visibility into the status of the request and uncertainty about whether a valid ticket will be ready when they reach the station.

Final Customer Problem Statement: I am a metro passenger who needs a ticket for my journey. I'm trying to book a valid ticket quickly through the Service Catalog, but the process can involve queues, manual fare calculation, and coordination with station staff because tickets are issued manually, which makes me feel frustrated and uncertain about whether my ticket will be ready on time.

Problem Statement Element	Description
I am	A metro passenger (commuter, student, tourist, or group organizer) who uses ServiceNow to request services.
I'm trying to	Book a valid metro ticket for a chosen route and date quickly, without standing in a queue.

Problem Statement Element	Description
But	Manual booking, unclear fare calculation, and approval or coordination for group tickets slow the process.
Because	Tickets are prepared manually and there is no single standardized request with automatic fare and ticket generation.
Which makes me feel	Frustrated, uncertain about the fare, and unsure whether my ticket will be ready on time.

2.2 Empathy Map Canvas

The empathy map focuses on the passenger as the primary user of the ServiceNow metro ticket process. The passenger wants a quick and transparent booking experience, an accurate fare, and instant proof of the ticket.

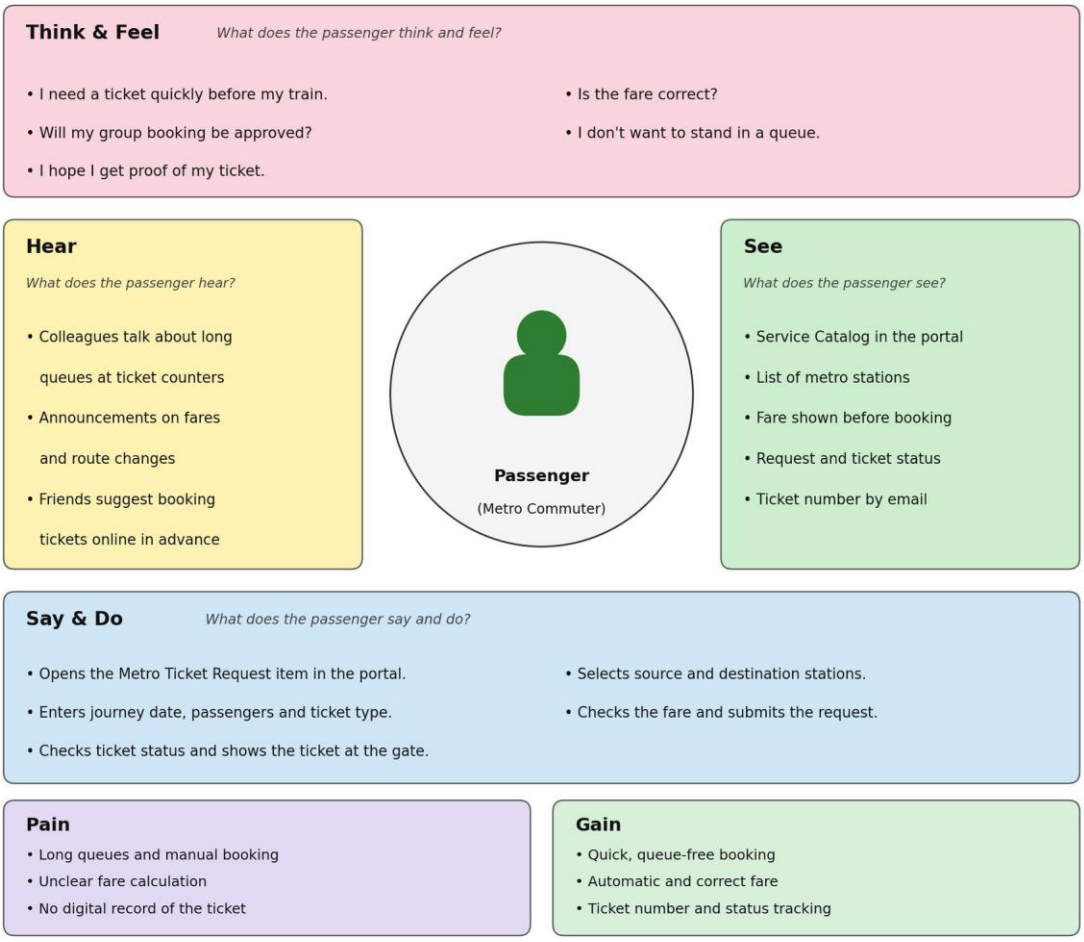


Figure 1. Empathy Map – Metro Passenger / Ticket Requester

- Needs: fast ticket booking, correct fare calculation, and transparent request and ticket tracking.
- Pain points: queues, manual fare calculation, waiting for approval on group bookings, and no digital ticket record.
- Expected gain: quick queue-free booking, automatic fare, a uniquely numbered digital ticket, and a better travel experience.

2.3 Brainstorming

Brainstorming was used to identify practical ideas that could reduce manual effort and improve the ticket booking experience. The ideas were grouped into request management, fare calculation, approval, ticket generation, notifications, and tracking.

Idea	Importance	Feasibility
Service Catalog metro ticket request form	High	High
Automatic fare calculation from station data	High	High
Automatic ticket generation with unique ticket number	High	High
Approval for group bookings (more than 5 passengers)	Medium	High
Ticket status tracking and automatic expiry	High	High
Email / status notifications with ticket details	Medium	High
QR code and payment gateway integration	Medium	Low

Selected idea: automate metro ticket generation using a ServiceNow Service Catalog item, a Flow Designer flow with fare calculation and conditional approval, and a custom Metro Ticket table. This idea was selected because it directly addresses the identified problem and can be implemented using standard ServiceNow capabilities. QR code and payment gateway integration were identified as valuable but were kept for future scope.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey describes the passenger experience from realizing the need to travel through request submission, validation and approval, ticket generation, and the journey itself. The key journey stages are Awareness, Request, Validation, Ticket Issue, and Journey.

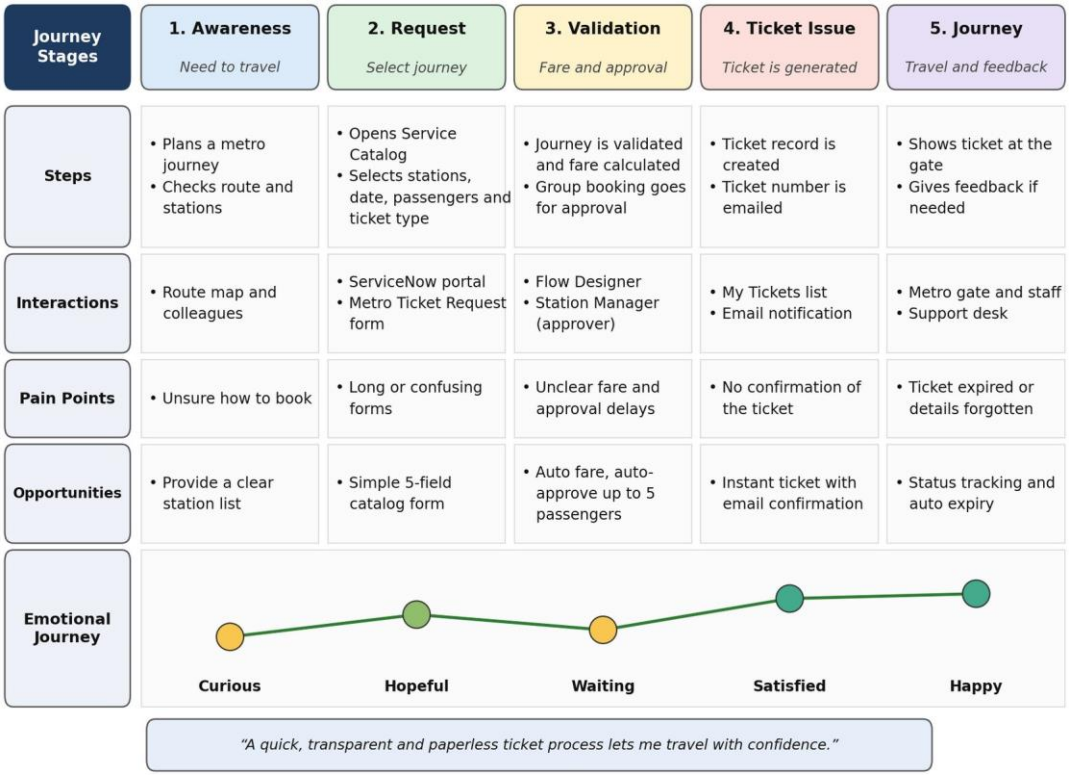


Figure 2. Customer Journey Map – Metro Ticket Request

The major opportunity areas are to provide a clear Service Catalog item, collect complete journey information, calculate the fare automatically, approve group bookings only when necessary, and confirm the ticket clearly to the passenger.

3.2 Solution Requirement

The following functional and non-functional requirements define the expected behavior and quality characteristics of the proposed ServiceNow solution.

Functional Requirements

FR No.	Functional Requirement	Sub Requirement / Expected Behavior
FR-1	Ticket Request Submission	Create a Service Catalog item; allow the passenger to submit a request; capture source, destination, journey date, passengers, ticket type, and payment mode.
FR-2	Request Processing	Create and manage REQ and RITM records; retain submitted values and link them to the catalog item.
FR-3	Journey Validation & Fare Calculation	Reject identical source and destination or invalid dates; calculate fare per passenger and total fare from station data.
FR-4	Dynamic Approval	Route group bookings (more than 5 passengers) to the Station Manager; allow approval/rejection; stop ticket generation when rejected.
FR-5	Ticket Generation	Create a Metro Ticket record with a unique ticket number, fare, validity, and status after a valid request.
FR-6	Tracking & Notifications	Allow passengers to view their tickets and status; send submission, approval/rejection, and ticket-generated notifications; expire old tickets automatically.

Non-functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	The Service Catalog item should be simple, clear, and easy for passengers to use.
NFR-2	Security	Passengers should see only their own tickets; only authorized roles may edit stations or tickets; ticket data must follow ACL controls.
NFR-3	Reliability	The flow should consistently process requests and generate exactly one ticket per approved request without losing request information.
NFR-4	Performance	Submission, validation, fare calculation, and ticket creation should complete within an acceptable response time.
NFR-5	Availability	The Service Catalog and ticket generation flow should be available whenever passengers need to book.
NFR-6	Scalability	The solution should support more stations, lines, ticket types, and users without major redesign.

3.3 Data Flow Diagram

The data flow shows how a metro ticket request moves through the ServiceNow platform. The passenger submits the request through the Service Catalog, ServiceNow creates the REQ and RITM records, the journey is validated and the fare is calculated using station data, group bookings are approved, and a Metro Ticket record is generated and notified to the passenger.

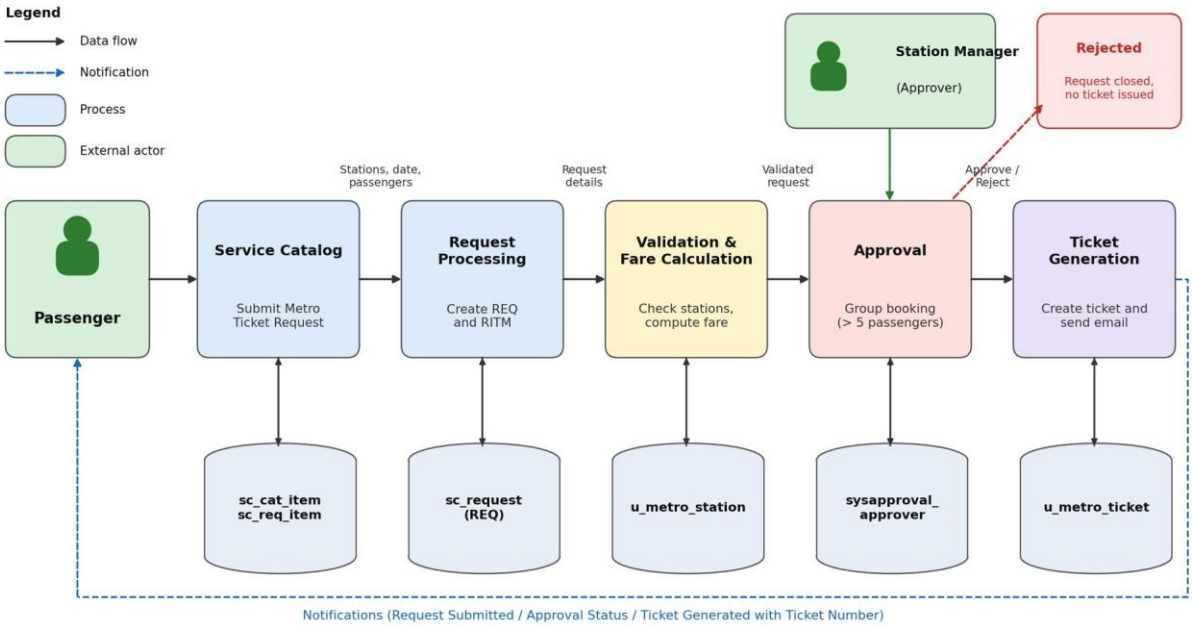


Figure 3. Data Flow Diagram – Metro Ticket Generating System

Stage	ServiceNow Record / Table	Purpose
Catalog Request	sc_cat_item	Metro Ticket Request catalog item
Request	sc_request (REQ)	Stores the overall request
Requested Item	sc_req_item (RITM)	Stores the requested item and catalog variables
Station Data	u_metro_station	Stores station code, name, line, and sequence used for fare
Approval	sysapproval_approver	Stores the group-booking approval decision
Ticket	u_metro_ticket	Stores the generated metro ticket, fare, validity, and status

Custom Table Design

Table	Key Fields
Metro Station (u_metro_station)	Station Code, Station Name, Line, Sequence (stop order used for fare), Active
Metro Ticket (u_metro_ticket)	Ticket Number (MTK prefix), Passenger, Source Station, Destination Station, Journey Date, Passengers, Ticket Type, Payment Mode, Fare per Passenger, Total Fare, Valid Until, Ticket Status, Requested Item

3.4 Technology Stack

Component	Technology	Purpose
User Interface	ServiceNow Service Portal / Service Catalog	Passenger submits the request and tracks tickets.
Application Logic	Service Catalog Item + Catalog Variables + Catalog Client Script	Captures Source, Destination, Journey Date, Passengers, Ticket Type, and Payment Mode; validates the form.
Workflow Automation	ServiceNow Flow Designer	Controls validation, conditional approval, ticket creation, and RITM closure.
Ticket Generation	Custom table Metro Ticket (u_metro_ticket)	Represents the generated digital ticket with number, fare, and validity.

Component	Technology	Purpose
Database / Records	sc_request, sc_req_item, sysapproval_approver, u_metro_station, u_metro_ticket	Stores request, requested item, approval, station, and ticket information.
Backend Automation	Business Rules + Scheduled Script Execution	Validates journey, calculates fare, sets validity, and expires old tickets.
Notifications & Security	Email Notifications, Events, ACLs, Roles	Communicates ticket status and restricts data access.
Deployment	ServiceNow Update Sets	Packages and migrates configuration between instances.
Infrastructure	ServiceNow PDI / Enterprise Instance	Hosts and executes the application and flow.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem-solution fit is achieved by replacing manual counter booking with a standardized ServiceNow request and an automated fare and ticket-generation process. The solution captures the information required to issue a ticket, validates the journey, calculates the fare, routes group bookings for authorization, and supports tracking.

Canvas Area	Project-Specific Definition
Customer Segments	Passengers (daily commuters, students, tourists, and group organizers) who use ServiceNow to request metro tickets.
Job-to-be-Done / Problems	Book a valid ticket for a chosen route and date quickly, know the exact fare, and keep a digital record of the ticket.
Problem Root Cause	Ticket issue depends on manual counters and calculation; without automation, queues and coordination delay the passenger.
Triggers	A work commute, class, event, sightseeing trip, or group outing creates the need for a metro ticket.
Available Solutions	Service Catalog item, station data, automatic fare calculation, conditional approval, ticket generation, tracking, and notifications.
Customer Constraints	Passengers need a correct fare and immediate confirmation; group bookings may need authorization.
Behaviour	Passenger selects stations and date, checks the fare, submits the request, waits for confirmation, and shows the ticket number at the gate.
Channels of Behaviour	Primary channel is the ServiceNow Service Portal / Service Catalog, supported by email notifications and the My Tickets list.
Your Solution	Standardized catalog item + automatic fare calculation + conditional group approval + unique Metro Ticket generation.

4.2 Proposed Solution

The proposed solution is a ServiceNow Metro Ticket Request catalog item connected to an automated validation, approval, and ticket-generation flow.

1. The passenger opens the ServiceNow Service Catalog and selects the Metro Ticket Request item.
2. The request form captures Source Station, Destination Station, Journey Date, Number of Passengers, Ticket Type, and Payment Mode.
3. ServiceNow creates the Request (REQ) and Requested Item (RITM) records.
4. The Flow Designer flow validates the journey and calculates the fare per passenger and the total fare using the station data.

5. If the number of passengers is greater than five, the flow routes the request to the Station Manager for approval; other requests are approved automatically.
6. After approval, the flow creates a Metro Ticket record with a unique ticket number, fare, validity, and status Generated.
7. Business Rules and notifications support backend processing; the passenger receives an email with the ticket details, and the RITM is closed as complete.
8. A scheduled job marks tickets as Expired after the journey date, and the configuration is captured in an Update Set so it can be migrated and validated in another instance.

Parameter	Proposed Solution
Problem to be solved	Queues, manual fare calculation, and coordination delays during metro ticket booking.
Idea / Solution	ServiceNow Service Catalog item with automatic fare calculation, conditional approval, and ticket generation.
Novelty / Uniqueness	A single standardized request connects catalog data, fare logic, approval, ticket generation, tracking, and deployment configuration.
Customer Satisfaction	Faster and more transparent booking with an exact fare and a uniquely numbered ticket.
Business Value	Reduces counter workload and standardizes the ticket lifecycle.
Scalability	Additional stations, lines, ticket types, fare rules, users, and approval rules can be added on the ServiceNow platform.

4.3 Solution Architecture

The solution architecture connects the passenger-facing Service Catalog with the ServiceNow data layer, flow automation, backend logic, ticket generation, notifications, and the Update Set deployment process.

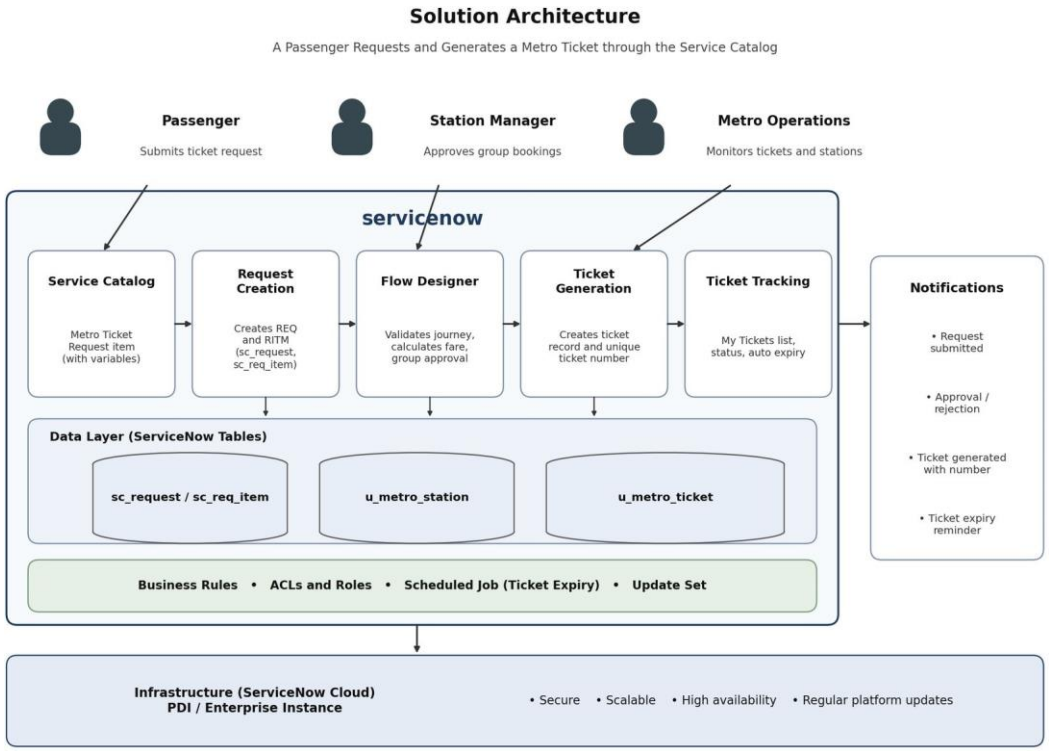


Figure 4. Solution Architecture – ServiceNow Metro Ticket Generating System

Component	Purpose
Passenger / Requestor	Initiates the metro ticket request.

Component	Purpose
Service Catalog	Provides the standardized request item and captures required variables.
ServiceNow Data Layer	Stores REQ, RITM, station, and ticket information.
Approval	Station Manager reviews and approves/rejects group bookings.
Flow Designer	Controls the automated validation, approval, and ticket generation sequence.
Business Rules	Support backend logic such as journey validation, fare calculation, and ticket validity.
Metro Ticket (u_metro_ticket)	Represents the generated digital ticket issued to the passenger.
Notifications	Communicate important request, approval, and ticket status.
Update Sets	Package and migrate configured ServiceNow changes.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was organized into four planned sprints with eight user stories and a total of 23 planned story points. The plan covers requirement analysis, Service Catalog configuration, data architecture, fare calculation, approval automation, ticket generation, Business Rules, notifications, testing, and Update Set deployment.

Sprint	User Story	Epic	Story Points	Priority	Planned Scope
Sprint-1	US-01	Requirement Analysis & Planning	2	High	Access the standardized Metro Ticket Request catalog item.
Sprint-1	US-02	Service Catalog	3	High	Capture Source, Destination, Journey Date, Passengers, Ticket Type, and Payment Mode.
Sprint-1	US-03	Data Architecture	3	High	Create station and ticket tables and relate them to REQ and RITM.
Sprint-2	US-04	Validation & Fare	3	High	Validate the journey and calculate the fare automatically.
Sprint-2	US-05	Workflow & Approval	3	High	Approve or reject group bookings; stop rejected requests.
Sprint-3	US-06	Ticket Generation & Business Rules	3	High	Generate a unique Metro Ticket and close the RITM.
Sprint-3	US-07	Notifications & Expiry	2	Medium	Send ticket, approval, and rejection updates; expire old tickets.
Sprint-4	US-08	Testing, Security & Deployment	4	High	Test, secure with ACLs, package, migrate, and validate using an Update Set.

Sprint Schedule

Sprint	Story Points	Duration	Planned Scope
Sprint-1	8	6 Days	Requirement analysis, Service Catalog, variables, and station/ticket data structure.
Sprint-2	6	6 Days	Journey validation, fare calculation, and approval automation.
Sprint-3	5	6 Days	Ticket generation, Business Rules, notifications, and ticket expiry.
Sprint-4	4	6 Days	Testing, security, Update Set, migration, and final validation.

Planned total story points: 23. Planned average velocity: 5.75 story points per sprint.

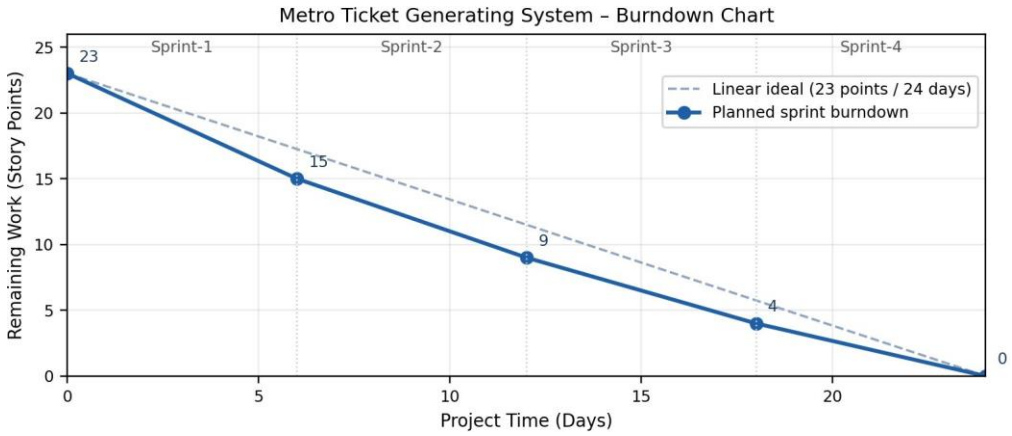


Figure 5. Project Planning / Burndown Representation

Checkpoint	Ideal Remaining Story Points	Actual Remaining
Project Start	23	Completed during project execution
End of Sprint-1	15	Completed during project execution
End of Sprint-2	9	Completed during project execution
End of Sprint-3	4	Completed during project execution
End of Sprint-4 / Completion	0	Completed during project execution

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Performance validation for this project focuses on correct and consistent processing of the metro ticket request through the Service Catalog, REQ, RITM, fare calculation, approval, ticket generation, automation, and Update Set stages. The testing confirms that the configured flow processes the request without missing records or flow breaks.

S.No.	Parameter	Validation	Result
1	Service Catalog Item	Catalog item loads correctly and displays the required request fields.	Pass
2	Request Data Capture	Source, Destination, Journey Date, Passengers, Ticket Type, and Payment Mode are captured.	Pass
3	REQ and RITM Processing	REQ and RITM are created and submitted values are retained.	Pass
4	Fare Calculation	Fare per passenger and total fare match the fare rule for the selected stations.	Pass
5	Approval Workflow	Group bookings reach the Station Manager and approval is recorded before ticket generation.	Pass
6	Ticket Generation	A single Metro Ticket with a unique number is created after a valid or approved request.	Pass
7	Automation & Data Integrity	Configured Business Rules and the scheduled job work and project configuration is captured for deployment.	Pass

Functional validation summary

Test Area	Expected Outcome	Status
Service Catalog & Request Submission	Request item is accessible and required details can be submitted.	Pass
REQ & RITM Processing	REQ and RITM are generated and retain the submitted catalog variables.	Pass
Validation & Fare Calculation	Invalid journeys are blocked and the correct fare is calculated.	Pass
Approval & Ticket Generation	Approval is recorded and approved requests create a Metro Ticket.	Pass
Business Rule / Automation	Configured backend automation executes as intended.	Pass
Update Set & Deployment Readiness	Configuration is captured and prepared for migration.	Pass
End-to-End Validation	Complete lifecycle works from request submission through ticket generation.	Pass

Sample test cases

TC ID	Scenario	Input / Condition	Expected Result	Status
TC-01	Valid single ticket	Central → University, 2 passengers, Single Journey	Fare Rs. 25 × 2 = Rs. 50; ticket generated; RITM Closed Complete	Pass
TC-02	Return ticket fare	Central → Airport, 2 passengers, Return Journey	Fare Rs. 45 per passenger; total Rs. 180	Pass
TC-03	Same source and destination	Central → Central	Error message shown; no ticket created	Pass
TC-04	Past journey date	Journey Date earlier than today	Submission blocked by catalog client script	Pass
TC-05	Group booking approval	8 passengers	Approval sent to Station Manager; ticket created only after approval	Pass
TC-06	Group booking rejected	Approver rejects the request	RITM closed incomplete; no ticket; rejection email sent	Pass
TC-07	Ticket notification	Ticket generated	Email contains ticket number, route, fare, and validity	Pass
TC-08	Ticket expiry	Journey date has passed	Scheduled job sets Ticket Status = Expired	Pass
TC-09	Access control	Another passenger opens the ticket	Read access denied; only owner and Metro Admin can view	Pass

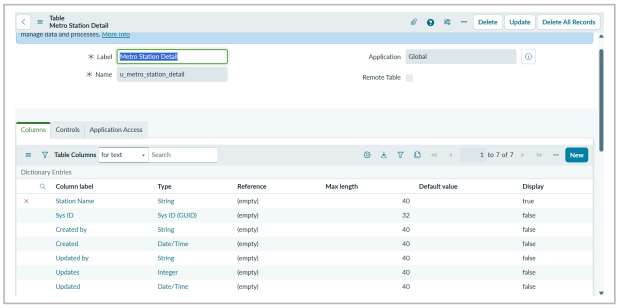
7. RESULTS

7.1 Output Screenshots

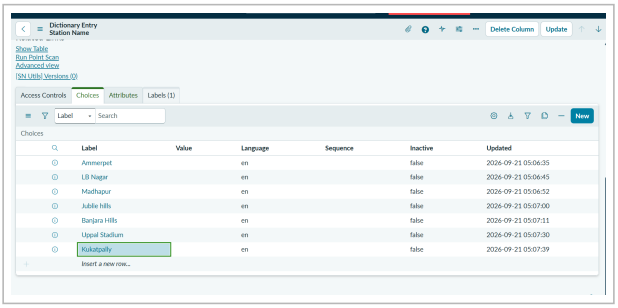
The following evidence points show the implemented ServiceNow configuration and working results. The screenshots are placed below the corresponding project outputs.

Output 1 – Metro Station Table and Records

ServiceNow custom table used to store the metro station details and configured station records.



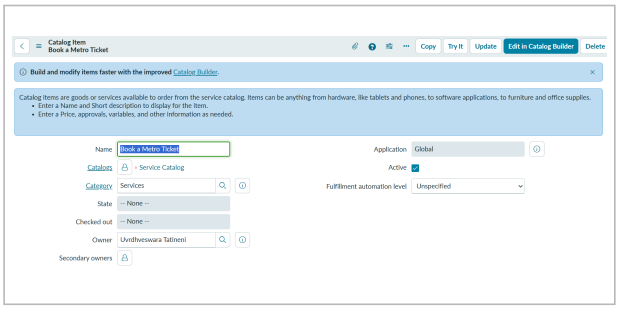
Metro Station Detail table



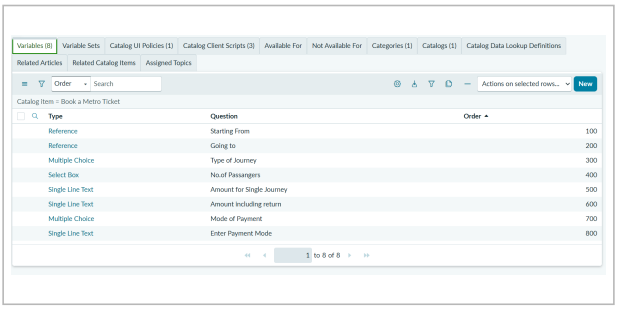
Configured station records

Output 2 – Service Catalog Item and Variables

The Book a Metro Ticket catalog item and its configured booking variables are shown below.



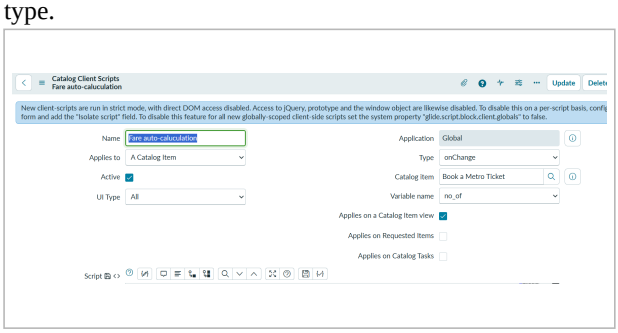
Book a Metro Ticket catalog item



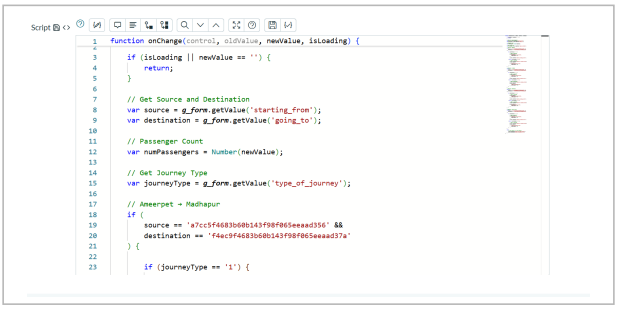
Catalog variables

Output 3 – Fare Auto-Calculation

The active onChange Catalog Client Script calculates the fare using the selected stations, passenger count, and journey type.



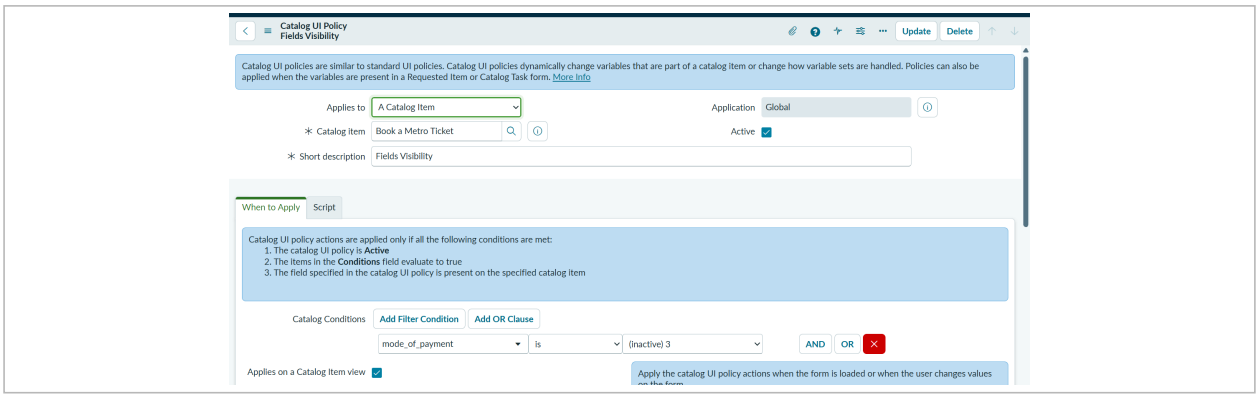
Fare auto-calculation configuration



Fare calculation script

Output 4 – Catalog UI Policy

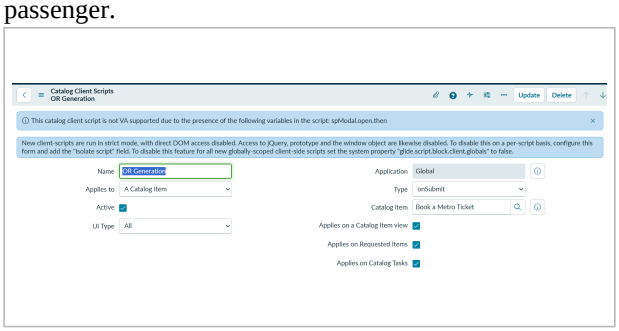
The Catalog UI Policy controls field visibility based on the selected payment mode.



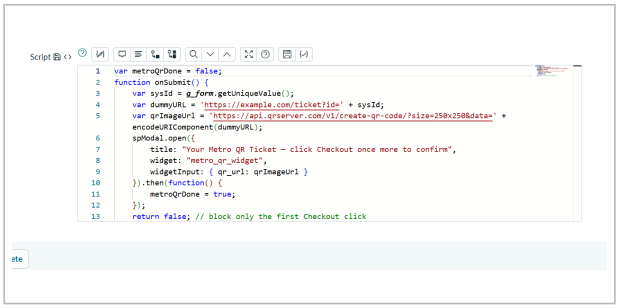
Fields Visibility – Catalog UI Policy

Output 5 – QR Generation

The onSubmit Catalog Client Script invokes the Metro QR widget and displays the generated QR ticket to the passenger.



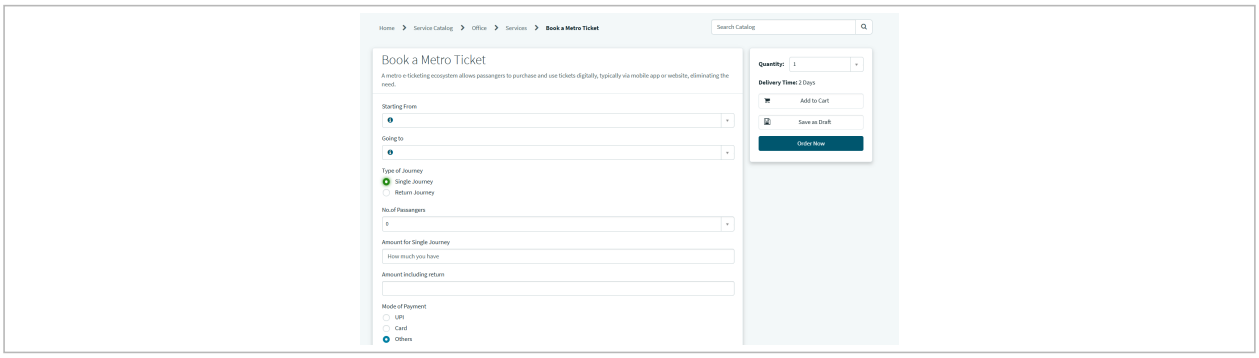
OR Generation configuration



QR generation script

Output 6 – Metro Ticket Request Form

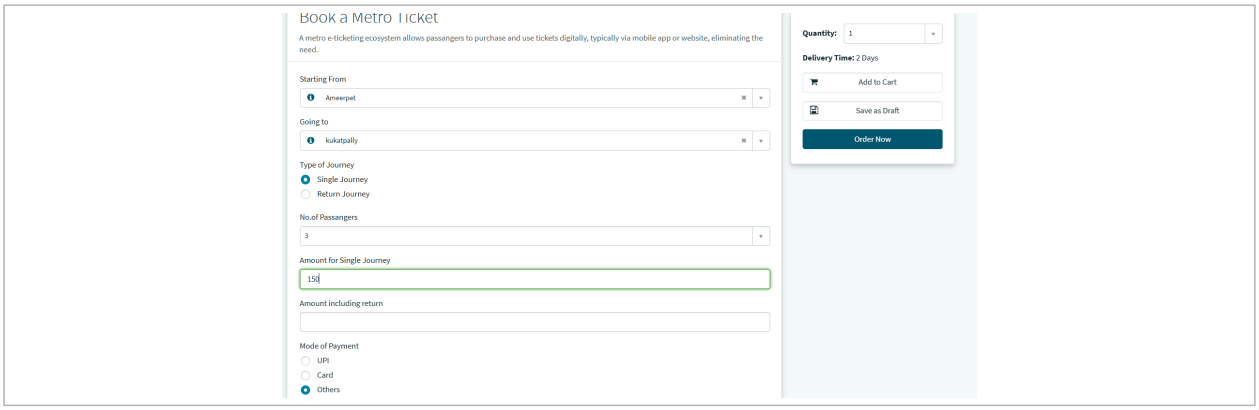
The Service Portal form captures the passenger journey and payment details before the ticket request is submitted.



Book a Metro Ticket – Service Portal form

Output 7 – Automatic Fare Result

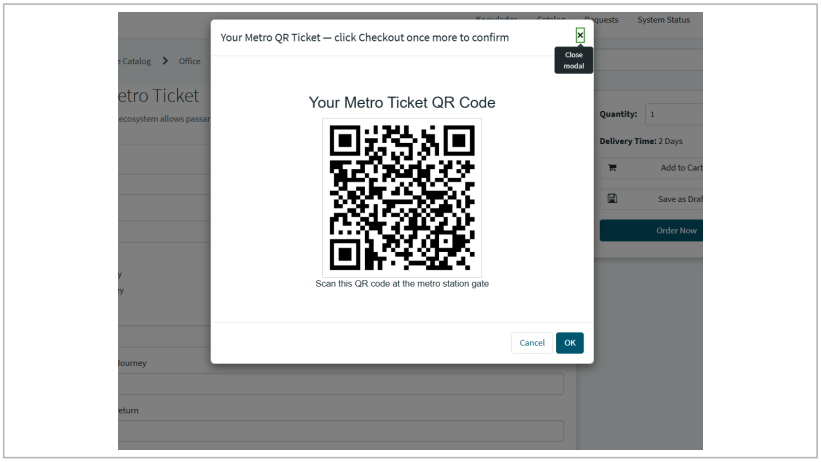
For the demonstrated journey, the system calculates the fare automatically and displays the result in the catalog form.



Automatic fare calculation result

Output 8 – Generated Metro QR Ticket

The generated Metro QR ticket is displayed to the passenger for use at the metro station gate.



Your Metro Ticket QR Code

Result summary

Requirement	Evidence	Outcome
FR-1 Ticket Request Submission	Output 2 and Output 6	Catalog item and booking form are configured and accessible.
FR-2 Station Data & Variables	Output 1 and Output 2	Station records and catalog variables are configured.
FR-3 Fare Calculation	Output 3 and Output 7	Fare calculation script and calculated fare are demonstrated.
FR-4 Dynamic Field Behavior	Output 4 and Output 6	Payment-mode based field visibility is configured.
FR-5 QR Ticket Generation	Output 5 and Output 8	QR generation script and QR ticket are demonstrated.

Requirement	Evidence	Outcome
FR-6 Tracking & Notifications	Output 7 and Output 9	Ticket status visible and email sent to the passenger
Deployment Readiness	Output 10	Configuration captured in the Update Set

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / Limitations
Standardized, paperless metro ticket process.	The solution depends on correct ServiceNow configuration and permissions.
Reduces queues and manual coordination at counters.	Payment gateway is not integrated; the payment mode is only recorded.
Automatic and consistent fare calculation from station data.	Group-booking approval delays can occur if the approver does not act promptly.
Generates a uniquely numbered digital ticket for every valid request.	The ticket is a digital record; QR scanning and gate integration are not included.
Provides request history, ticket status visibility, and automatic expiry.	The fare rule is a simplified demonstration model, not a real operator fare table.
Secures passenger data through roles and ACLs.	Notifications and integrations may require additional configuration for a production environment.
Can be extended to more stations, lines, ticket types, and approval rules.	The project does not include external APIs or machine-learning functionality.

9. CONCLUSION

The Metro Ticket Generating System project successfully demonstrates how ServiceNow can be configured to standardize and automate a metro ticket request lifecycle. The solution begins with a structured Service Catalog form, creates REQ and RITM records, validates the journey and calculates the fare, routes group bookings for approval, and generates a uniquely numbered Metro Ticket for the passenger.

The project also demonstrates backend automation through Business Rules and a scheduled job, request and ticket data handling through ServiceNow tables, access control through ACLs, and configuration migration using Update Sets. Overall, the solution improves process consistency, reduces manual effort, and gives passengers a clearer and faster path from ticket request to a valid digital ticket.

10. FUTURE SCOPE

- Integrate a payment gateway (UPI, cards, wallets) so that the ticket is generated only after successful payment.
- Generate a QR code on each ticket and integrate with gate validation systems for entry and exit scanning.
- Integrate with a real metro operator system for live station, fare, and service-status data.
- Add cancellation and refund workflows with configurable cancellation windows.
- Support smart-card top-up and concession categories such as student, senior citizen, and daily pass.
- Add multi-line routes with interchange stations and dynamic peak/off-peak fare rules.
- Add dashboards and Performance Analytics for daily ticket sales, peak hours, and approval turnaround time.
- Provide a mobile-friendly experience, richer email / in-app notifications, and a Virtual Agent for booking and FAQs.

11. APPENDIX

A. Source Code

The project primarily uses ServiceNow configuration, Flow Designer actions, catalog variables, custom tables, and Business Rules. The following scripts are used in the project.

Business Rule 1 – Validate Metro Journey (u_metro_ticket, Before Insert)

```
(function executeRule(current, previous /*null when async*/) {
    // Source and destination must be different stations
    var src = current.getValue('u_source_station');
    var dst = current.getValue('u_destination_station');
    if (src == dst) {
        gs.addErrorMessage('Source and destination cannot be the same.');
        current.setAbortAction(true);
    }
})(current, previous);
```

Business Rule 2 – Calculate Metro Fare (u_metro_ticket, Before Insert)

```
(function executeRule(current, previous /*null when async*/) {
    var src = new GlideRecord('u_metro_station');
    var dst = new GlideRecord('u_metro_station');
    if (src.get(current.getValue('u_source_station')) &&
        dst.get(current.getValue('u_destination_station'))) {

         var stations  = Math.abs(parseInt(dst.getValue('u_sequence'), 10) -
                                  parseInt(src.getValue('u_sequence'), 10));
        // Rs. 10 base fare + Rs. 5 per station, maximum Rs. 60
        var perPerson = Math.min(60, 10 + (5 * stations));
        var trips = (current.getValue('u_ticket_type') == 'return') ? 2 : 1;
        var pax = parseInt(current.getValue('u_passengers'), 10);

        current.u_fare_per_passenger = perPerson;
        current.u_total_fare = perPerson * trips * pax;

        // Ticket is valid until the end of the journey date
        var day = current.getValue('u_journey_date');
        current.u_valid_until = new GlideDateTime(day + ' 23:59:59');
    }
})(current, previous);
```

Scheduled Script Execution – Expire Metro Tickets (Daily)

```
var gr = new GlideRecord('u_metro_ticket');
gr.addQuery('u_ticket_status', 'generated');
gr.addQuery('u_valid_until', '<', gs.nowDateTime());
gr.query();
while (gr.next()) {
    gr.setValue('u_ticket_status', 'expired');
    gr.update();
}
```

Flow Designer – Metro Ticket Generation Flow (summary)

- Trigger: Service Catalog (Metro Ticket Request).
- Get Catalog Variables → look up source and destination stations.
- If Source = Destination → set RITM to Closed Incomplete and stop.
- If Number of Passengers > 5 → Ask for Approval (Station Manager); if rejected → set RITM to Closed Incomplete, send rejection email, and stop.

- Create Record in Metro Ticket (fare and validity are set by the Business Rules).
- Send email "Metro Ticket Generated" → update RITM state to Closed Complete.

B. GitHub & Project Demo Link

GitHub Link: [Add your GitHub repository link here]

Project Demo Link: View Metro Ticketing Project Demo

C. Core ServiceNow Records Used

Record / Table	Meaning	Project Use
sc_cat_item	Catalog Item	Metro Ticket Request form.
sc_request	REQ	Overall request record.
sc_req_item	RITM	Requested item and submitted variables.
u_metro_station	Metro Station	Station code, name, line, and sequence used for fare calculation.
sysapproval_approver	Approval	Stores the group-booking approval decision.
u_metro_ticket	Metro Ticket	Generated ticket with number, fare, validity, and status.

Sample station data and fare rule

Fare per passenger = Rs. 10 base fare + Rs. 5 × number of stations travelled (maximum Rs. 60). A Return Journey is charged twice, and Total Fare = Fare per passenger × Number of passengers. This rule is a simplified model for demonstration only.

Seq.	Station	Code	Seq.	Station	Code
1	Central	CTL	5	Tech Park	TCP
2	Market Square	MKT	6	Stadium	STD
3	Old Town	OLT	7	City Mall	CML
4	University	UNI	8	Airport	ARP

D. Acceptance Criteria

- Passenger can submit the Metro Ticket Request through the Service Catalog.
- The catalog captures Source Station, Destination Station, Journey Date, Number of Passengers, Ticket Type, and Payment Mode.
- A REQ and corresponding RITM are created and linked correctly.
- A request with the same source and destination, or a past journey date, is not accepted.
- The fare per passenger and total fare are calculated automatically from the station data.
- Group bookings (more than 5 passengers) are routed to the Station Manager for an approve/reject decision.
- A valid or approved request creates one Metro Ticket with a unique ticket number and status Generated.
- A rejected request does not generate a ticket.
- The passenger receives a notification with the ticket details, and expired tickets are marked Expired automatically.
- The configured solution can be packaged and migrated using an Update Set.

GitHub Link: https://github.com/rishibopyapati/metro-ticket-generating-system